Memory-Centric Agentic Inference — cycles 10-12

2026-05-11

Contents

Memory-Centric Agentic Inference — cycles 10-12

Abstract

Cycles 10-12 converted the post-architecture package from a set of synthetic models into a more constrained and inspectable research package. Cycle 10 repaired `M-COMP-1`, the compression and offload boundary model, by removing an unsupported queue-threshold preservation claim. Cycle 11 built `M-PROTO-1`, a trace-replay runtime prototype that turns the architecture options into an executable object-registry and placement-policy loop. Cycle 12 built `M-CALIB-1`, a sourced calibration map that connects the synthetic models to public hardware, interconnect, storage, benchmark, and systems evidence without retroactively relabeling synthetic coefficients as measured facts.

The main result is a narrower and stronger form of the central thesis. The package now supports memory-centric architecture where retained state value exceeds coordination overhead, but it distinguishes three evidence levels: validated synthetic mechanism, public evidence for conventional/KV/prefix-cache memory management, and deferred evidence for durable agent trajectory reuse, provenance validation cost, semantic-cache correctness, and contention-sensitive remote memory behavior.

Introduction

The run investigates whether future agentic large-language-model infrastructure should be organized around memory objects and their movement, placement, reuse, compression, and lifetime, rather than around arithmetic throughput alone. Earlier cycles established the foundation: a workload and memory-object taxonomy, symbolic lifetime and cost models, a simulator, scheduling abstraction comparisons, an architecture proposal, a trace schema, a queueing model, and a compression/offload boundary.

Cycles 10-12 focused on making that package harder to overclaim. The work addressed three questions:

1. Can compression/offload be claimed to help avoid queueing reversals, or only to help

with capacity, movement, storage locality, provenance, and safety?
2. Can the proposed architecture be expressed as a concrete runtime loop over trace events rather than only as prose and tables?
3. Which parts of the synthetic package are supported by public sources, and which remain deferred because public evidence is missing or too indirect?

The three architecture options remain unchanged:

- **Option A**, conventional request/model/KV-centric serving.
- **Option B**, a memory-object-aware runtime that tracks object-local reuse, provenance, invalidation, and pointer-preserving representation.
- **Option C**, a trajectory/DAG-aware memory fabric that treats branch, verifier, durable workspace, and trajectory state as first-class scheduling and placement inputs.

Methodology

The reporter pass used the supplied cycle sessions and workspace artifacts as source material. The relevant session records were:

Cycle	Role	Session ID	Main content
10	researcher	`c5a431ed-7c49-4914-af87-3172d3baa949`	Repair brief for `M-COMP-1` queue attribution.
10	worker	`1d742a27-2d88-4671-a062-df60cb0a0703`	Implemented object-level compression queue attribution and narrowed claim.
10	auditor	`54d64f9e-1c42-47f2-831b-ab23f25bd375`	Validated repaired `M-COMP-1`.
11	researcher	`e0a1a7c0-53a7-4d68-a59d-0717f2a5af43`	Brief for trace-replay runtime prototype.
11	worker	`9afa5972-ac95-49d5-9d47-102724a68e28`	Implemented `M-PROTO-1`.
11	auditor	`b9cde876-0eb5-4bca-96ec-60a543530ed4`	Validated runtime prototype.
12	researcher	`9af586a7-6dff-4d57-aec9-2d5ad6f8052e`	Brief for sourced calibration map.
12	worker	`bcd4268a-81d3-4e4e-b01b-b89c2d10b200`	Built calibration tables, figures, and references.
12	auditor	`0d1e5f6c-7c54-4903-b423-1e4d74c3f1e2`	Validated calibration map after a plotter patch.

No independent audit was performed during this reporting pass. Audit findings are reported as upstream results.

Results

Cycle 10: Compression Queue Attribution Repair

Cycle 10 continued M-COMP-1 after the previous audit found that compression queue-help labels were not supported once net_queue_effect_proxy > 0 was required. The researcher brief framed the issue as an attribution problem: workload-level aggregation could either hide a real positive object-level effect or fabricate one by mixing unrelated overhead and relief terms.

The worker updated scripts/evaluate_compression_strategies.py, scripts/plot_compression_strategies.py, and memory-centric-agentic/compression_model.md. The new table data/compression_object_queue_interactions.csv records queue effects at (workload_class, object_class, strategy) granularity. Workload-level helps_* labels in data/compression_queue_interactions.csv are now allowed only when selected positive object-level evidence exists.

The repaired rule is:

```
QueueRelief(i,s) > ReconstructionCost(i,s) + MetadataCost(i,s)
```

for a valid object i and compression/offload strategy s whose queue participates in an M-QUEUE-1 reversal threshold.

The regenerated outputs showed:

Output	Result
compression_strategy_scores.csv	210 data rows
compression_best_strategy_by_object.csv	35 data rows
compression_workload_summary.csv	6 data rows
compression_safety_failures.csv	29 data rows
compression_object_queue_interactions.csv	146 data rows
compression_queue_interactions.csv	19 data rows
Selected positive object queue-help rows	0
Workload-level helps_* rows	0
Queue-harm rows	16

The auditor validated the narrowed-claim path. The decision was that M-COMP-1 no longer supports a positive queue-threshold preservation claim under the current synthetic trace and coefficients. It does support compression/offload as capacity, movement, local-storage, provenance-preservation, and safety machinery. Unsafe lossy summarization remains rejected for correctness-sensitive replay and provenance state.

Cycle 11: Runtime Prototype

Cycle 11 started M-PROTO-1, a toy runtime prototype for object registry and tier-placement decisions. The goal was not production scheduling or calibrated performance. The goal was to test whether the validated trace v2 interface could drive a concrete policy loop that reconstructs memory-object state and reproduces the Option A/B/C architecture boundary.

The worker implemented:

- memory-centric-agentic/runtime_prototype.md
- scripts/runtime_prototype.py

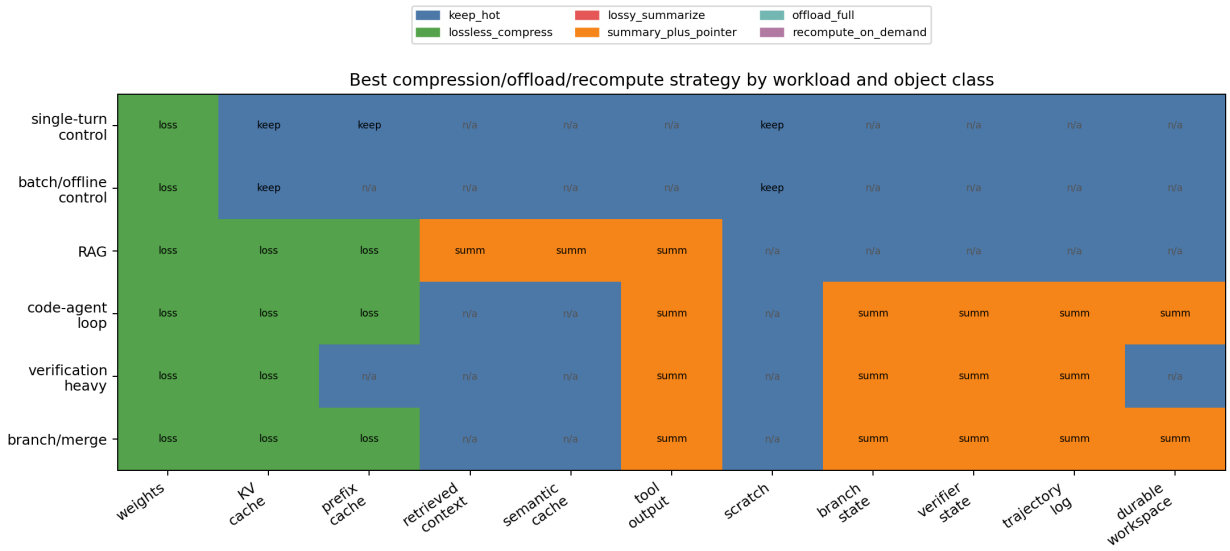

Figure 1: compression/offload strategy choices by workload and memory-object class

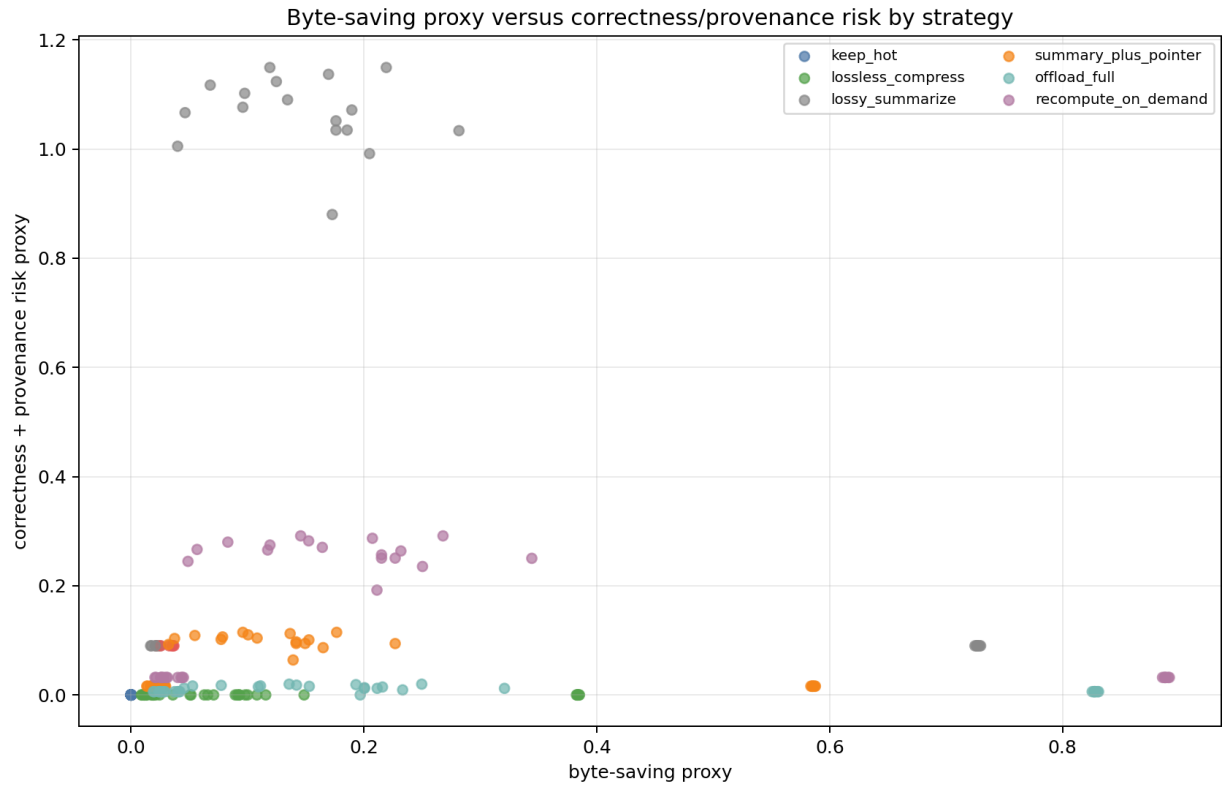

Figure 2: byte-saving proxy versus correctness/provenance risk for compression strategies

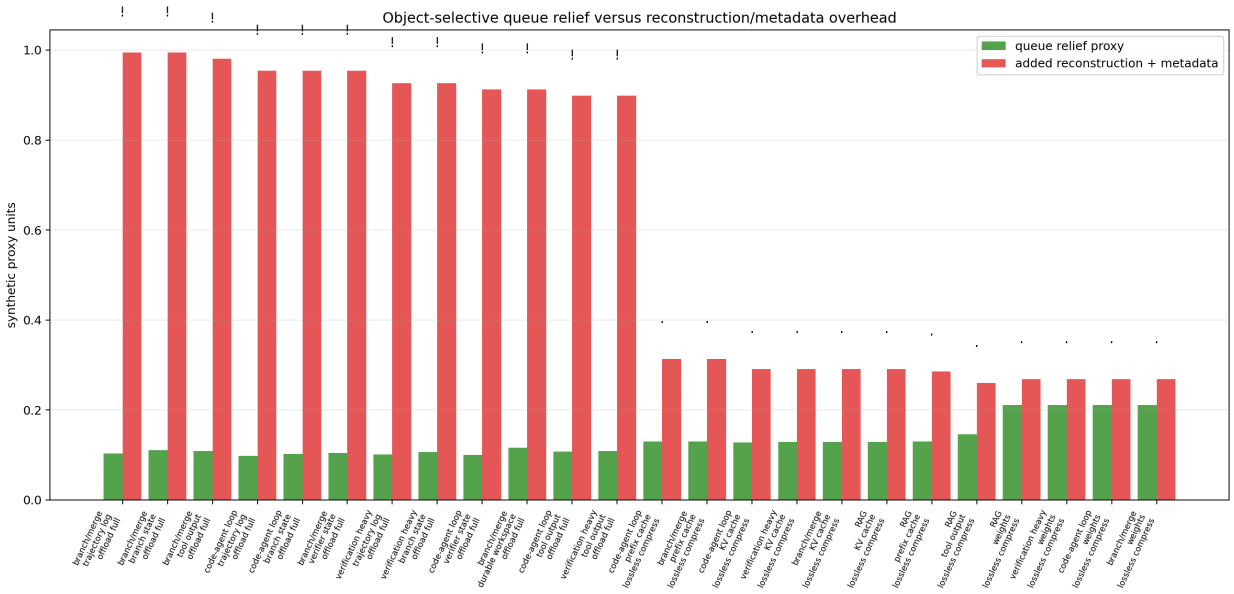

Figure 3: object-selective queue relief versus reconstruction and metadata overhead

- `scripts/plot_runtime_prototype.py`
- `data/runtime_registry_snapshots.csv`
- `data/runtime_policy_decisions.csv`
- `data/runtime_workload_summary.csv`
- `data/runtime_ablation_results.csv`
- `data/runtime_failure_cases.csv`

The runtime consumes trace v2 and prior model outputs, including `data/agentic_trace_events_v2.csv`, `data/trace_object_lifetimes.csv`, `data/queueing_architecture_winners.csv`, `data/compression_best_strategy_by_object.csv`, `data/compression_object_queue_interactions.csv`, and `data/architecture_policy_matrix.csv`.

The object registry tracks fields such as object class, workload class, size, tier, lifetime, reuse count, reuse distance, correctness sensitivity, provenance ID, source version, invalidation signal, trajectory node, branch ID, verifier ID, durability horizon, merge state, placement decision, retention decision, compression strategy, and eviction decision.

The runtime reproduced the expected architecture boundary:

Workload	Runtime option	Expected option	Match
single-turn chat control	Option A	Option A	yes
batch/offline control	Option A	Option A	yes
RAG	Option B	Option B	yes
code-agent loop	Option C	Option C	yes
verification-heavy	Option C	Option C	yes
multi-agent branch/merge	Option C	Option C	yes

The audit reported:

Check	Result
Workloads	6
Registry object classes	11
Policy decisions	66
Failure rows	32
Unsafe lossy compression selections	0
Unsupported queue-help claims	0

The ablation table tested whether runtime choices depend on causal memory fields rather than workload labels. Hiding provenance and reuse collapses RAG from Option B to Option A. Hiding branch, verifier, and durable fields collapses code-agent, verification-heavy, and multi-agent branch/merge workloads from Option C toward Option B. Hiding all memory-causal fields collapses non-control workloads to Option A.

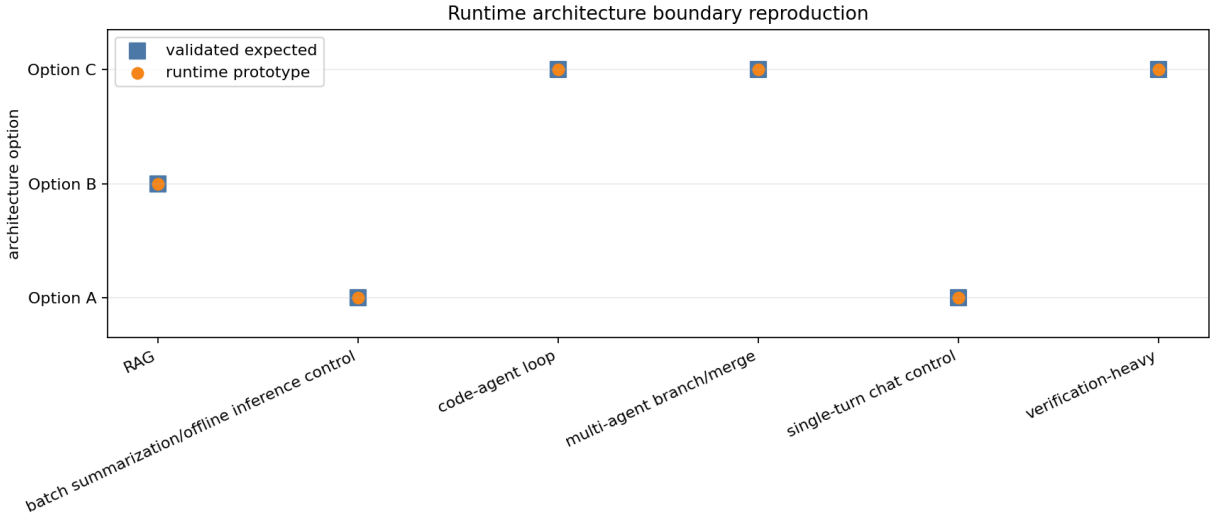

Figure 4: runtime-selected architecture option by workload

The auditor validated `M-PROTO-1` and noted that the prototype makes the architecture proposal executable as a small registry and policy loop while preserving synthetic-only claims and compression/queue safety constraints.

Cycle 12: Sourced Calibration Map

Cycle 12 built `M-CALIB-1`, a references-backed calibration map for memory-tier constants and workload evidence. The cycle was explicitly not a calibrated rerun of prior models. Its purpose was to connect validated synthetic artifacts to public evidence and to mark missing evidence as deferred rather than guessed.

The worker built:

- `REFERENCES.md`
- `memory-centric-agentic/calibration_map.md`
- `scripts/build_calibration_map.py`
- `scripts/plot_calibration_map.py`
- `data/calibration_memory_tiers.csv`

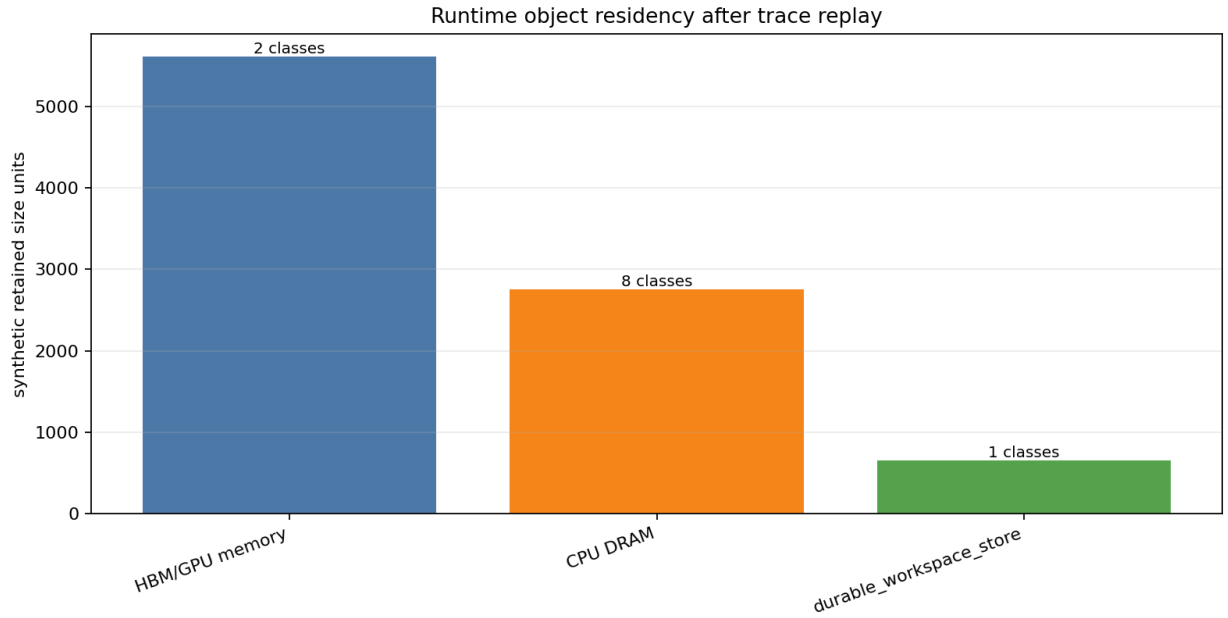

Figure 5: synthetic tier residency and retained object classes during trace replay

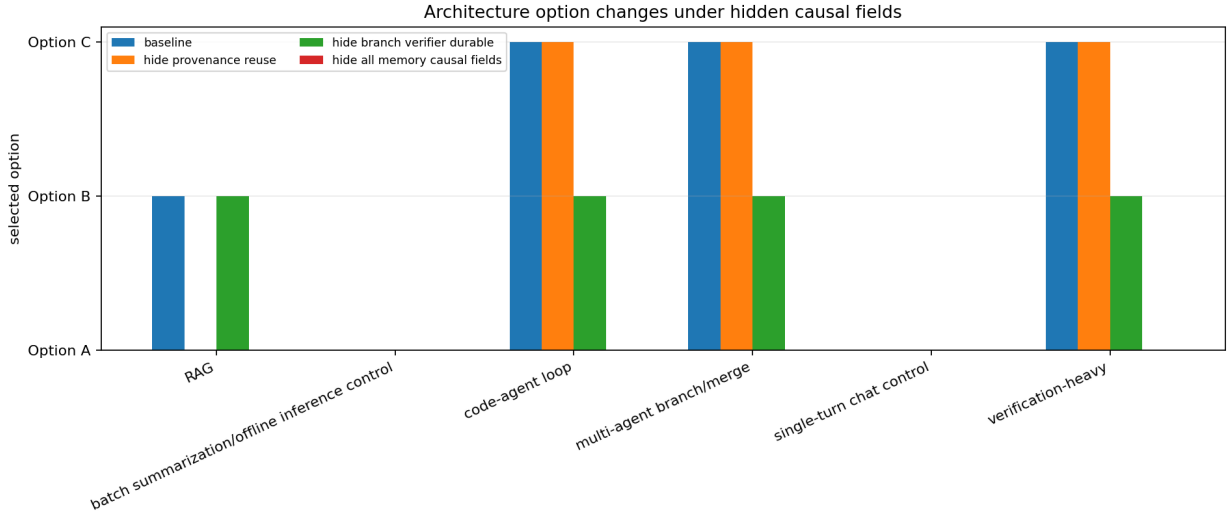

Figure 6: architecture-option changes under provenance, correctness, branch, verifier, and durable-field ablations

- `data/calibration_workload_evidence.csv`
- `data/calibration_deferred_constants.csv`
- `data/calibration_model_mapping.csv`
- `data/calibration_source_quality_summary.csv`

The calibration map distinguishes claim types:

- `sourced_range`
- `derived_from_source`
- `measured_in_prior_artifact`
- `synthetic_carryover`
- `deferred_public_evidence_missing`

The final artifacts include 14 numbered references, 16 memory-tier rows across 9 tier categories, 6 workload-evidence rows, 6 deferred constants, and 8 model-mapping rows. The auditor spot-checked key local reference claims against external sources, including NVIDIA H100/H200/DGX B200 pages, PCI-SIG PCIe 6.0, MLPerf Inference Datacenter, PagedAttention/vLLM, and vLLM prefix caching.

The memory-tier table covers:

- HBM/GPU memory
- GPU-to-GPU fabric
- host CPU DRAM
- PCIe host link
- CXL/pooled memory
- local NVMe
- NVMe-over-fabrics/remote storage
- durable workspace/object store
- semantic/prefix cache service

The calibration map keeps capability rows separate from deployment constants. For example, CXL and NVMe rows are treated as protocol or capability evidence when public latency, contention, energy, or production deployment behavior is unavailable.

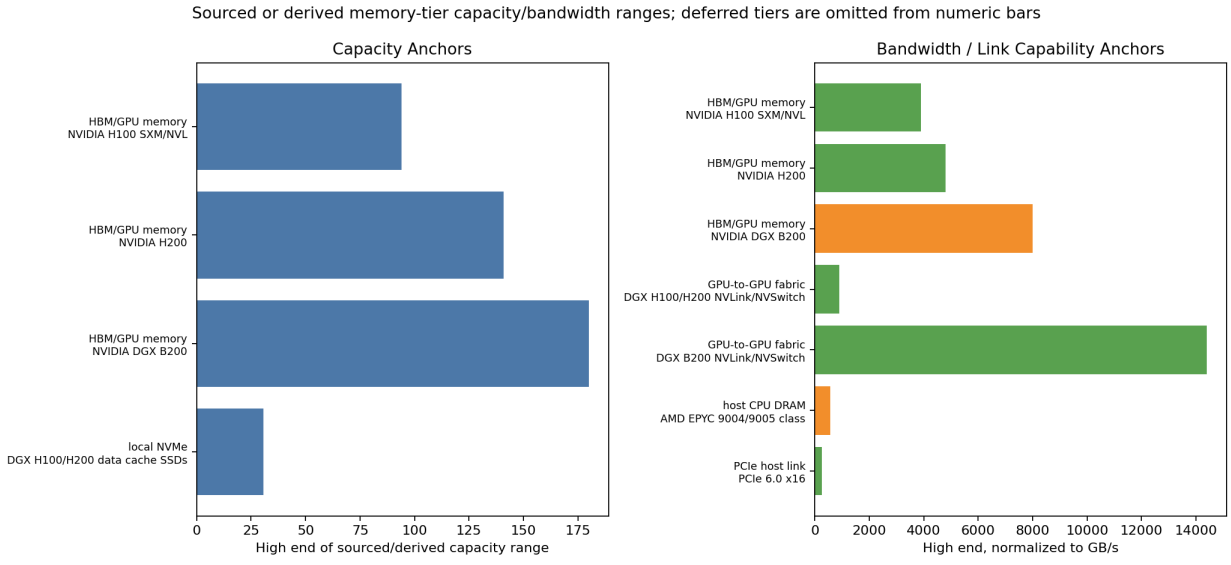

Figure 7: sourced and derived memory-tier capacity/bandwidth ranges

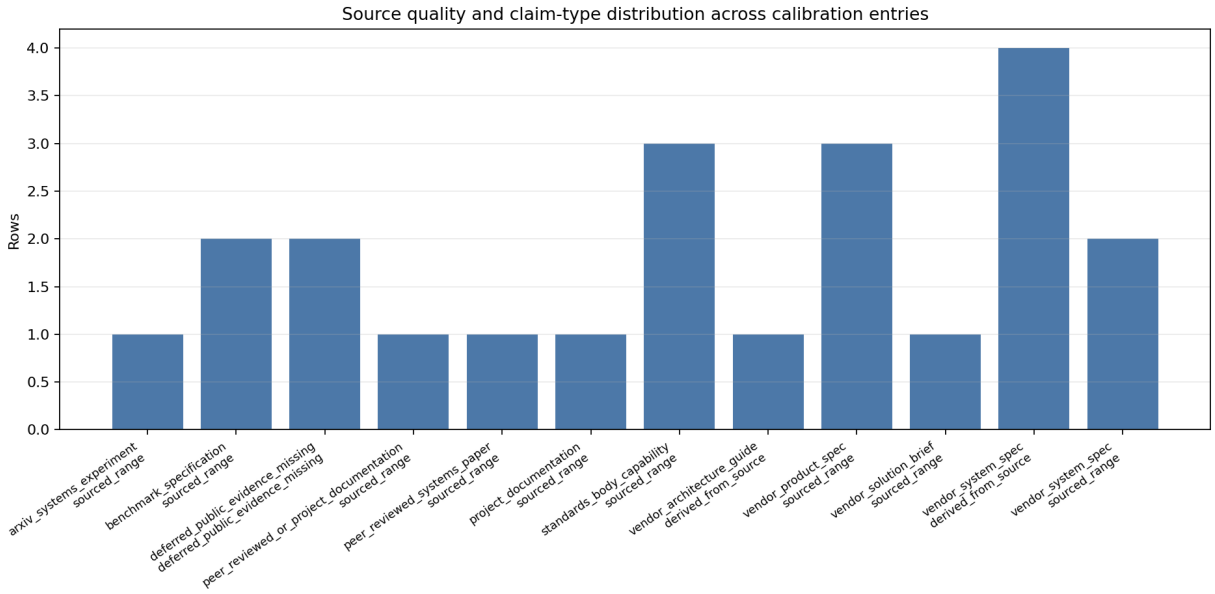

Figure 8: source quality and claim-type distribution across calibration entries

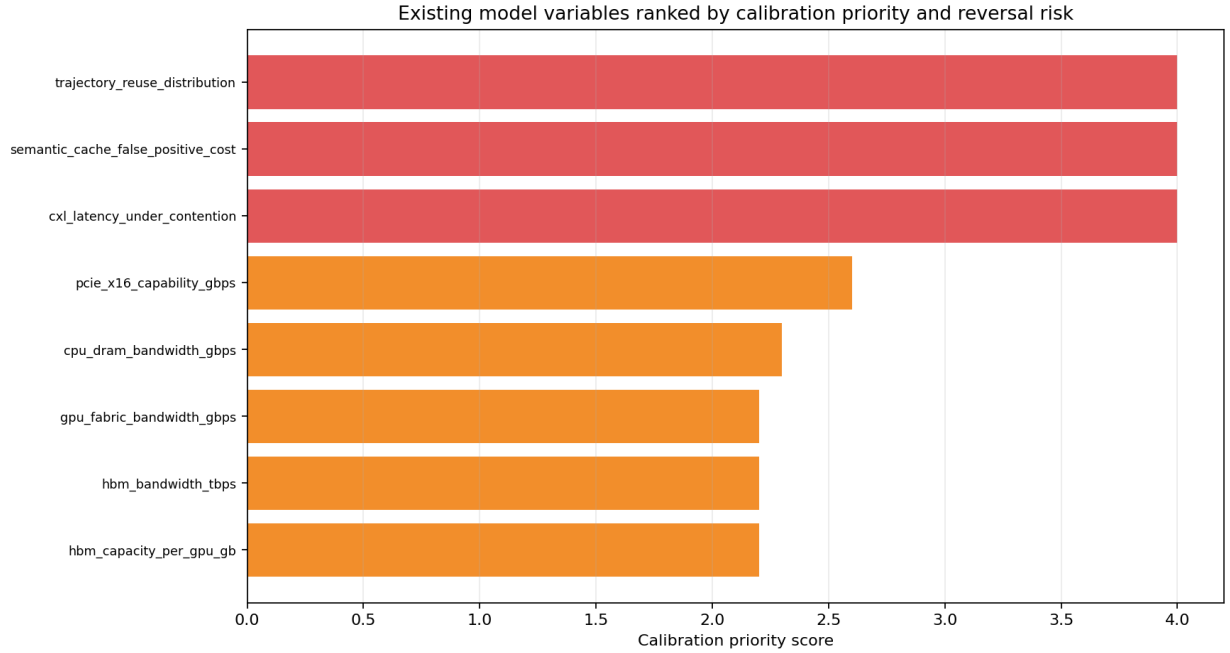

Figure 9: model variables ranked by calibration readiness and reversal risk

The deferred constants are now first-class outputs:

Deferred constant	Why it matters
Per-tier energy per byte moved or retained	Determines whether memory-centric placement reduces power and cost.
CXL memory latency under contention	Can reverse object or trajectory placement when pooled memory adds queueing delay.
Remote object-store latency distributions	Affects durable workspace replay and checkpoint cost.
Semantic-cache correctness and invalidation cost	Affects whether semantic reuse helps or harms RAG/object-aware runtimes.
Production agent trajectory reuse distributions	Affects whether Option C has measured retained-state value.
Provenance-validation overhead	Affects pointer-preserving compression and replay safety.

The auditor found one moderate defect: `scripts/plot_calibration_map.py` failed when regenerating a single figure with a relative `FIGURE_OUT=data/...` path. The auditor patched the script to normalize relative output paths against the workspace root. After the patch, builder, plotter, single-figure regeneration, `py_compile`, reference checks, figure checks, `promise_check`, and `org_check` passed.

Discussion

The cycle 10 repair changed the interpretation of compression. Before the repair, compression/offload appeared to help avoid queueing reversals in some workload-level classifications. After object-level attribution, no selected valid object-strategy pair had positive net queue effect under the current synthetic setup. The resulting conclusion is more limited: compression is still part of the memory-centric architecture, but because representation validity and provenance dominate, it is not currently evidence for preserving Option B/C under coordination overhead.

The cycle 11 prototype made the architecture proposal operational. The important result is not the synthetic score values. The important result is that trace-visible fields are sufficient to reconstruct an object registry and choose between Option A, Option B, and Option C without hard-coding only workload names. The ablations show which fields carry the decision: provenance/reuse for RAG, and branch/verifier/durable/trajectory fields for agentic workloads.

The cycle 12 calibration map separated public evidence from synthetic inference. Public sources support the reality of conventional serving controls, KV-cache memory management, prefix-cache reuse, and hardware/interconnect/storage capability ranges. They do not yet provide clean production evidence for durable trajectory reuse, verifier-state retention value, semantic-cache correctness costs, provenance-validation overhead, or CXL/remote-memory latency under agentic contention.

Together, cycles 10-12 bound the thesis as follows:

```
memory-centric architecture is strongest when
retained memory-state value > coordination and validation overhead
```

That statement remains synthetic for durable agent trajectories, but it is now connected to an executable prototype and a sourced calibration map.

Conclusions and Recommendations

Cycles 10-12 validated three additions to the research package.

First, `M-COMP-1` is now defensible because it no longer claims queue-threshold help without positive object-level evidence. Compression/offload remains important for capacity, movement, local storage, provenance preservation, and safety.

Second, `M-PROTO-1` shows that the Option A/B/C architecture boundary can be expressed as a trace-replay runtime loop with an object registry, placement decisions, retention decisions, compression decisions, eviction decisions, ablations, and failure cases.

Third, `M-CALIB-1` provides the first cited bridge from the synthetic package to public evidence. It preserves the boundary between public facts, transparent derivations, synthetic carryover, and deferred missing evidence.

The recommended next milestone is `M-SEC-1`: security, provenance, isolation, and retention-risk analysis for durable agentic state. The cycle 12 auditor specifically recommended using deferred constants and provenance/correctness gaps as threat-model drivers, including semantic-cache false positives, stale tool outputs, cross-user prefix or semantic-cache leakage, durable workspace retention, trajectory-log replay, and provenance-pointer validation.

References

[1] NVIDIA, "NVIDIA H100 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-us/data-center/h100/

[2] NVIDIA, "NVIDIA H200 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-us/data-center/h200/

[3] NVIDIA, "NVIDIA DGX B200: The Foundation for Your AI Factory," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-gb/data-center/dgx-b200/

[4] NVIDIA, "Introduction to NVIDIA DGX H100/H200 Systems," NVIDIA DGX H100/H200 User Guide, accessed 2026-05-11. https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html

[5] PCI-SIG, "PCI Express 6.0 Specification," PCI-SIG, accessed 2026-05-11. https://pcisig.com/pci-express-6.0-specification

[6] NVM Express, "Specifications," NVM Express, accessed 2026-05-11. https://nvmexpress.org/specificatio

[7] Compute Express Link Consortium, "CXL Specification," Compute Express Link, accessed 2026-05-11. https://computeexpresslink.org/cxl-specification/

[8] MLCommons, "MLPerf Inference: Datacenter Benchmark," MLCommons, accessed 2026-05-11. https://mlperf.pw/benchmarks/inference-datacenter/index.html

[9] Woosuk Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," arXiv, 2023. https://arxiv.org/abs/2309.06180

[10] Intel, "Intel Xeon 6 Processors with MRDIMM — Solution Brief," Intel, accessed 2026-05-11. https://www.intel.com/content/www/us/en/content-details/919018/intel-xeon-6-processors-with-mrdimm-solution-brief.html

[11] AMD, "AMD EPYC 9005 Processor Architecture Overview," AMD, accessed 2026-05-11. https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/user-guides/58462_amd-epyc-9005-tg-architecture-overview.pdf

[12] vLLM Project, “Automatic Prefix Caching,” vLLM Documentation, accessed 2026-05-11. https://docs.vllm.ai/en/latest/design/prefix_caching/

[13] Sajal Regmi and Chetan Phakami Pun, “GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching,” arXiv, 2024. https://arxiv.org/abs/2411.05276

[14] CXL Consortium, “CXL Consortium Releases Compute Express Link 2.0 Specification,” Business Wire, 2020. https://www.businesswire.com/news/home/20201110005037/en/CXL-Consortium-Releases-Compute-Express-Link-2.0-Specification

Appendix: Implementation Details

Code Organization

Cycle 10 artifacts:

- `memory-centric-agentic/compression_model.md`
- `scripts/compression_model.wls`
- `scripts/evaluate_compression_strategies.py`
- `scripts/plot_compression_strategies.py`
- `data/compression_special_cases.csv`
- `data/compression_boundary_inequalities.csv`
- `data/compression_strategy_scores.csv`
- `data/compression_best_strategy_by_object.csv`
- `data/compression_workload_summary.csv`
- `data/compression_safety_failures.csv`
- `data/compression_object_queue_interactions.csv`
- `data/compression_queue_interactions.csv`
- `data/compression_strategy_matrix.png`
- `data/compression_safety_vs_savings.png`
- `data/compression_queue_relief.png`

Cycle 11 artifacts:

- `memory-centric-agentic/runtime_prototype.md`
- `scripts/runtime_prototype.py`
- `scripts/plot_runtime_prototype.py`
- `data/runtime_registry_snapshots.csv`
- `data/runtime_policy_decisions.csv`
- `data/runtime_workload_summary.csv`
- `data/runtime_ablation_results.csv`
- `data/runtime_failure_cases.csv`
- `data/runtime_architecture_boundary.png`
- `data/runtime_object_residency.png`
- `data/runtime_ablation_effects.png`
- `stale/runtime_placeholder_2026-05-11.txt`

Cycle 12 artifacts:

- `REFERENCES.md`
- `memory-centric-agentic/calibration_map.md`
- `scripts/build_calibration_map.py`
- `scripts/plot_calibration_map.py`
- `data/calibration_memory_tiers.csv`
- `data/calibration_workload_evidence.csv`

- data/calibration_deferred_constants.csv
- data/calibration_model_mapping.csv
- data/calibration_source_quality_summary.csv
- data/calibration_tier_bandwidth_capacity.png
- data/calibration_source_quality.png
- data/calibration_model_sensitivity_targets.png

Test and Audit Results

Cycle 10 audit result: VALIDATED.

- Wolfram regenerated 11 compression special cases and 5 boundary inequalities.
- Python evaluator regenerated 210 strategy score rows, 35 best-object rows, 6 workload summaries, 29 safety failures, 146 object queue rows, and 19 workload queue rows.
- Queue invariant passed: zero helps_* rows with nonpositive net queue effect.
- Three compression figures were nonblank.
- promise_check and org_check passed.

Cycle 11 audit result: VALIDATED.

- Runtime regenerated registry, policy, workload summary, ablation, and failure-case outputs.
- Six workloads and 11 object classes were covered.
- Expected Option A/B/C boundary matched for all workloads.
- RAG provenance/reuse ablation collapsed Option B to Option A.
- Branch/verifier/durable ablations collapsed agentic Option C cases toward Option B.
- Unsafe lossy compression selections were zero.
- Three runtime figures were nonblank.
- py_compile, promise_check, and org_check passed.

Cycle 12 audit result: VALIDATED.

- One plotter defect was patched by the auditor: relative FIGURE_OUT=data/... paths are now normalized against the workspace root.
- Builder regenerated 16 memory-tier rows, 6 workload-evidence rows, 6 deferred constants, and 8 model mappings.
- References were contiguous from [1] to [14].
- Independent checks found zero invalid references, zero uncited numeric calibrated rows, and zero placeholder rows.
- Three calibration figures were nonblank.
- py_compile, promise_check, and org_check passed.

File Counts

The updated workspace manifest records:

Category	Count
Scripts	24
Script lines	5,713
Markdown model/assumption/proposal files	14
CSV data/model files	54
Figures	24
Sub-topics completed or assessed	11

Cross-Reference Map

Source artifact	Consuming artifact	Role
`data/agentic_trace_events_v2.csv`	`scripts/simulate_queueing_overheads.py`	Supplies event rates, migration counts, live bytes, DAG width, verifier delay, and durable rates.
`data/queueing_reversal_thresholds.csv`	`scripts/evaluate_compression_strategies.py`	Defines when compression can help or hurt architecture reversals.
`data/compression_object_queue_interactions.csv`	`scripts/runtime_prototype.py`	Prevents unsupported queue-help claims in runtime policy output.
`data/compression_best_strategy_by_object.csv`	`scripts/runtime_prototype.py`	Supplies validated compression/offload choices while unsafe lossy cases remain blocked.
`data/agentic_trace_events_v2.csv`	`scripts/runtime_prototype.py`	Drives object-registry reconstruction and placement decisions.
`REFERENCES.md`	`scripts/build_calibration_map.py`	Validates public source references for calibration rows.
`data/calibration_model_mapping.csv`	future calibrated variants	Identifies which synthetic variables are ready for public-range substitution, capability-only treatment, or deferral.